

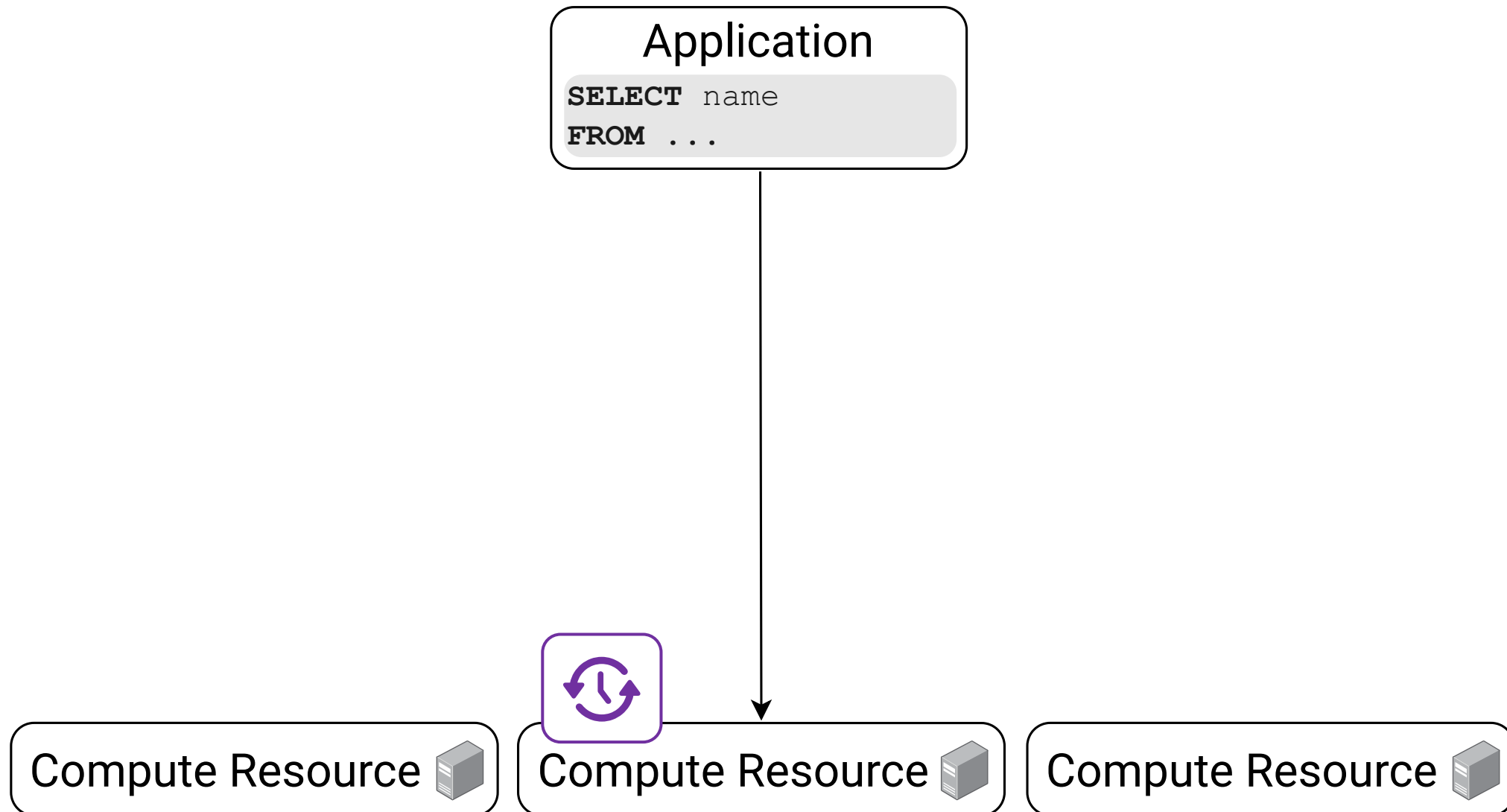
Design and Implementation of a Framework for Performance Characterization of Resource Sharing Mechanisms in Distributed Computing



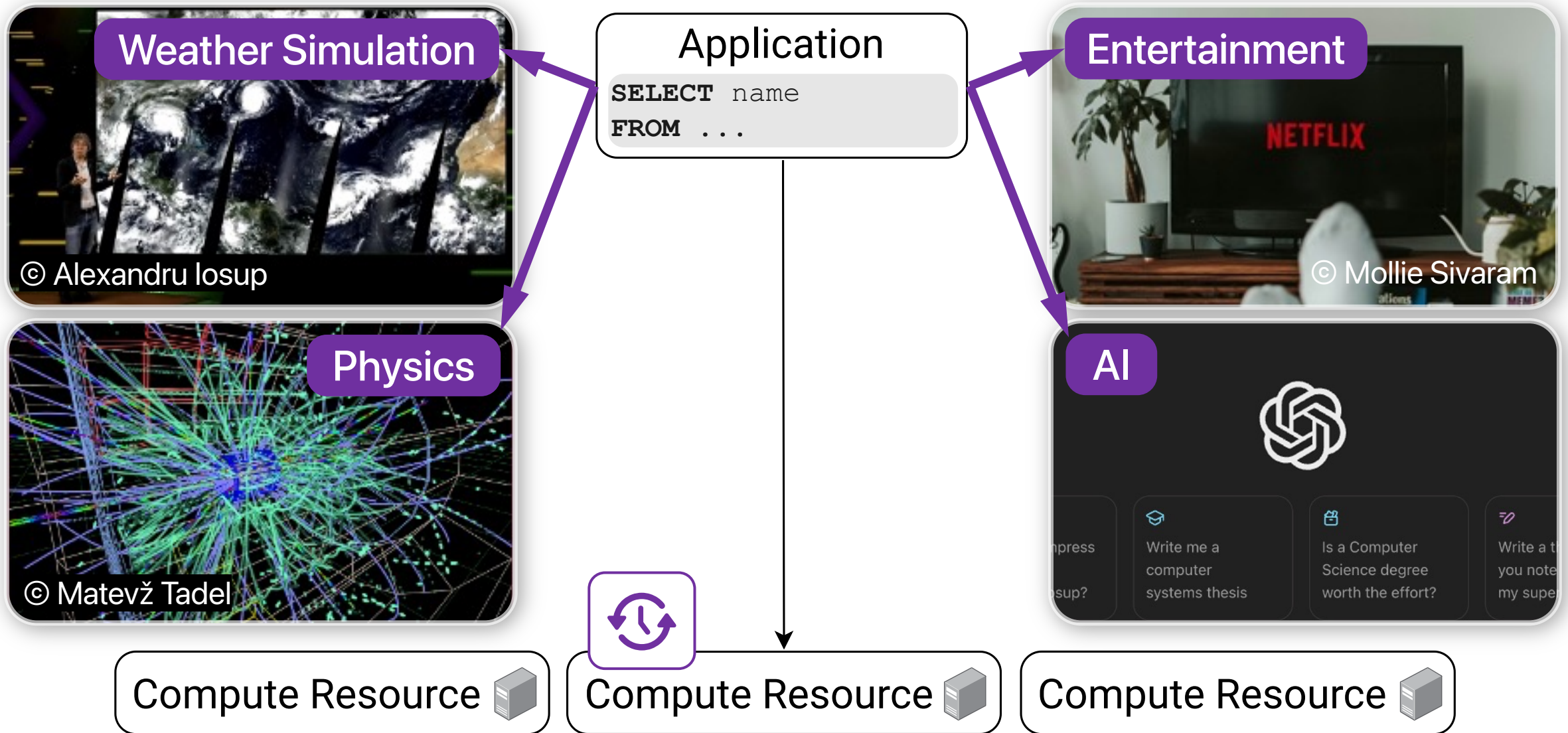
BSc Thesis by
Lennart K. M. Schulz

under supervision from
Prof. Dr. Ir. Alexandru Iosup and Sacheendra Talluri

at
Vrije Universiteit Amsterdam, NL



Context – *Where Are We?*

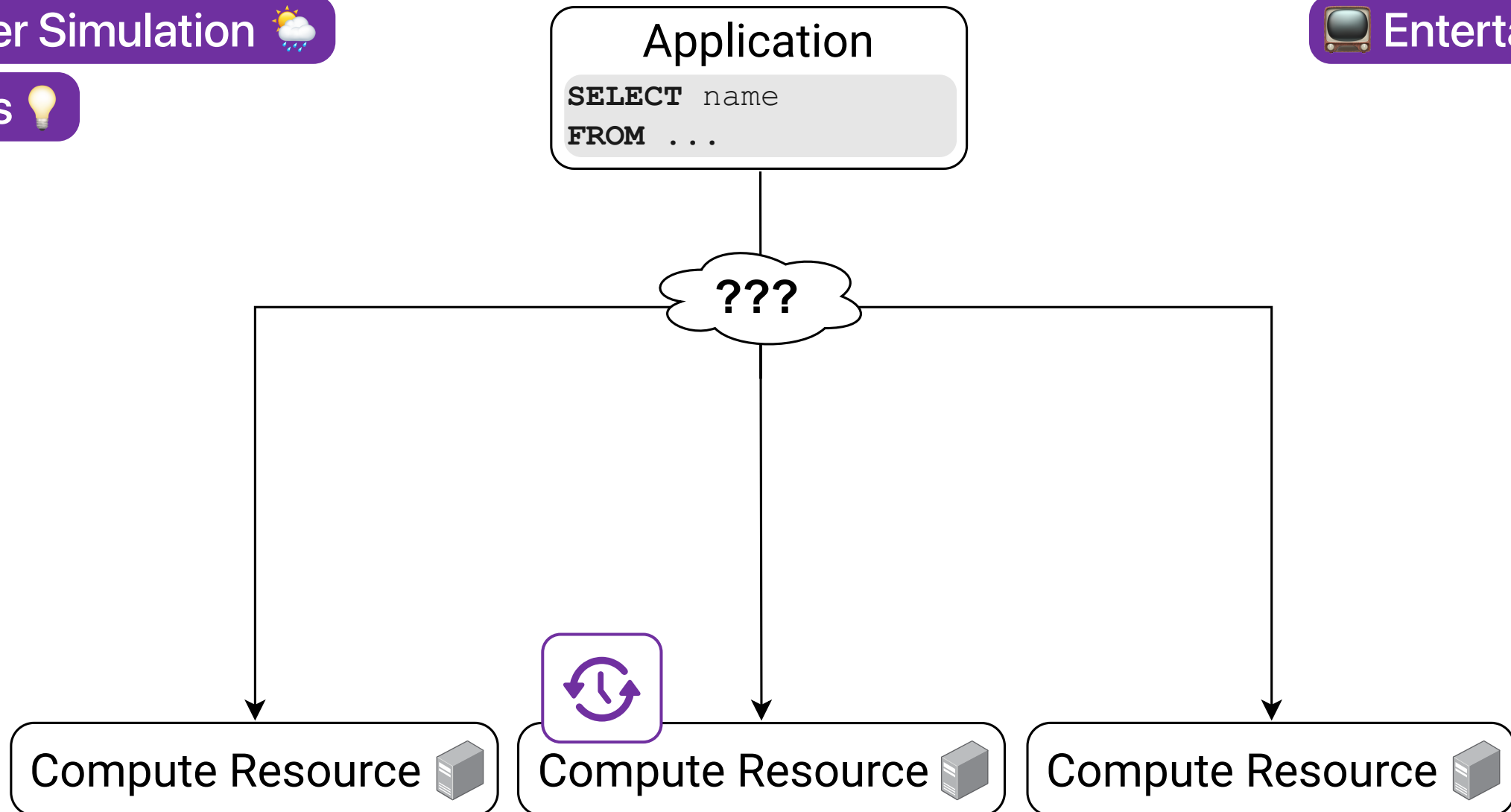


Weather Simulation 🌤️

Physics 💡

📺 Entertainment

🤖 AI

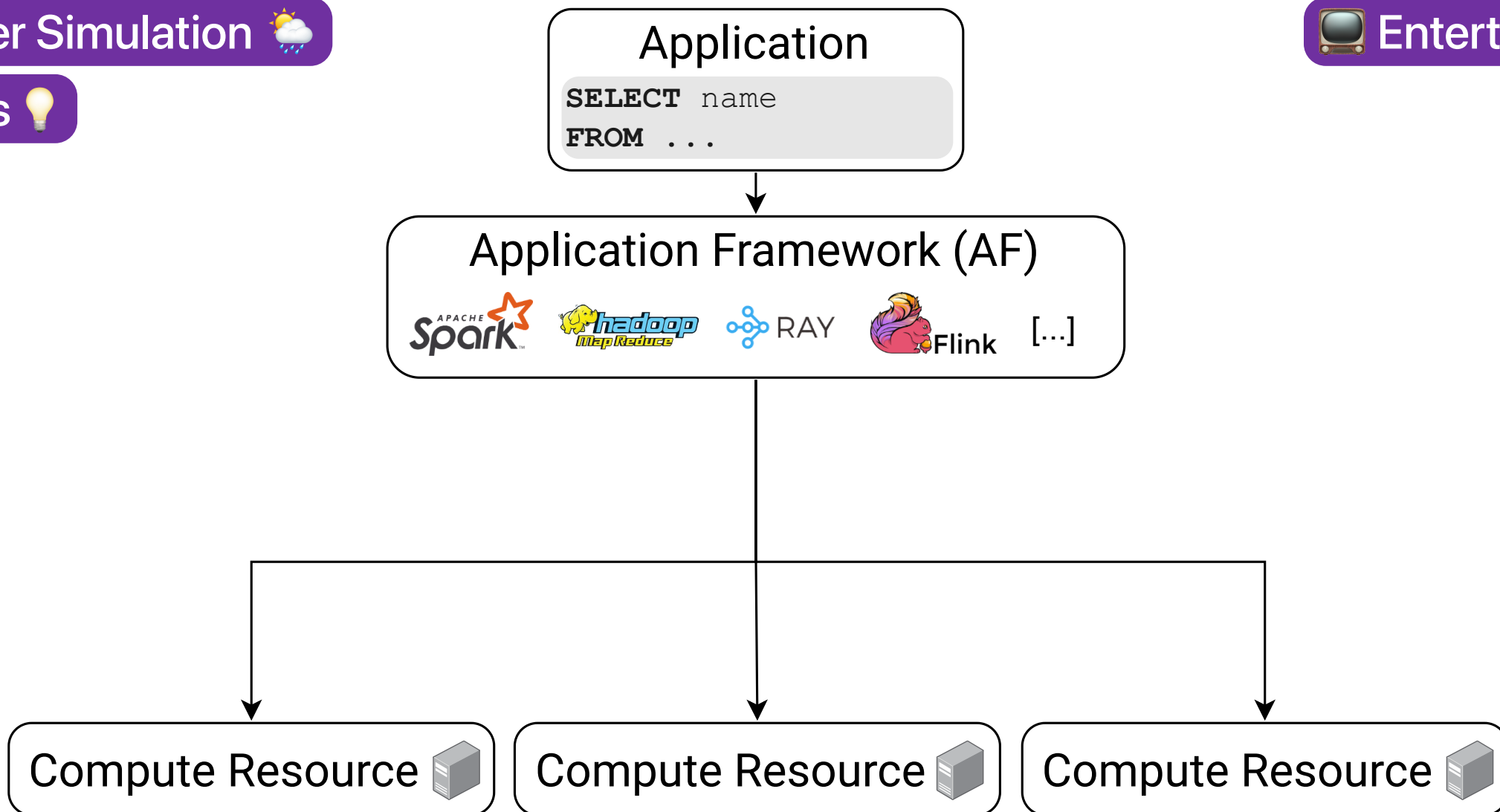


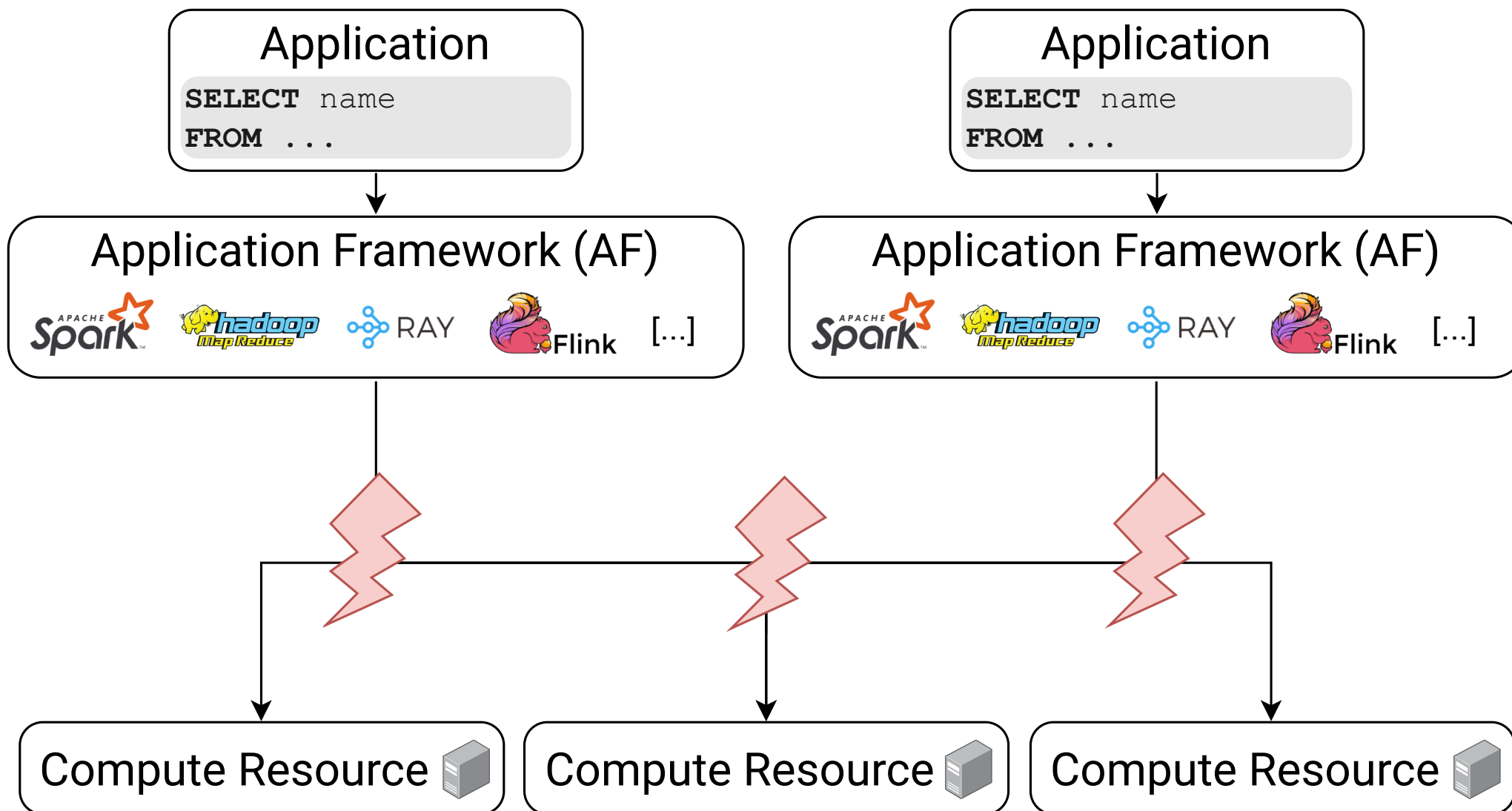
Weather Simulation 🌤️

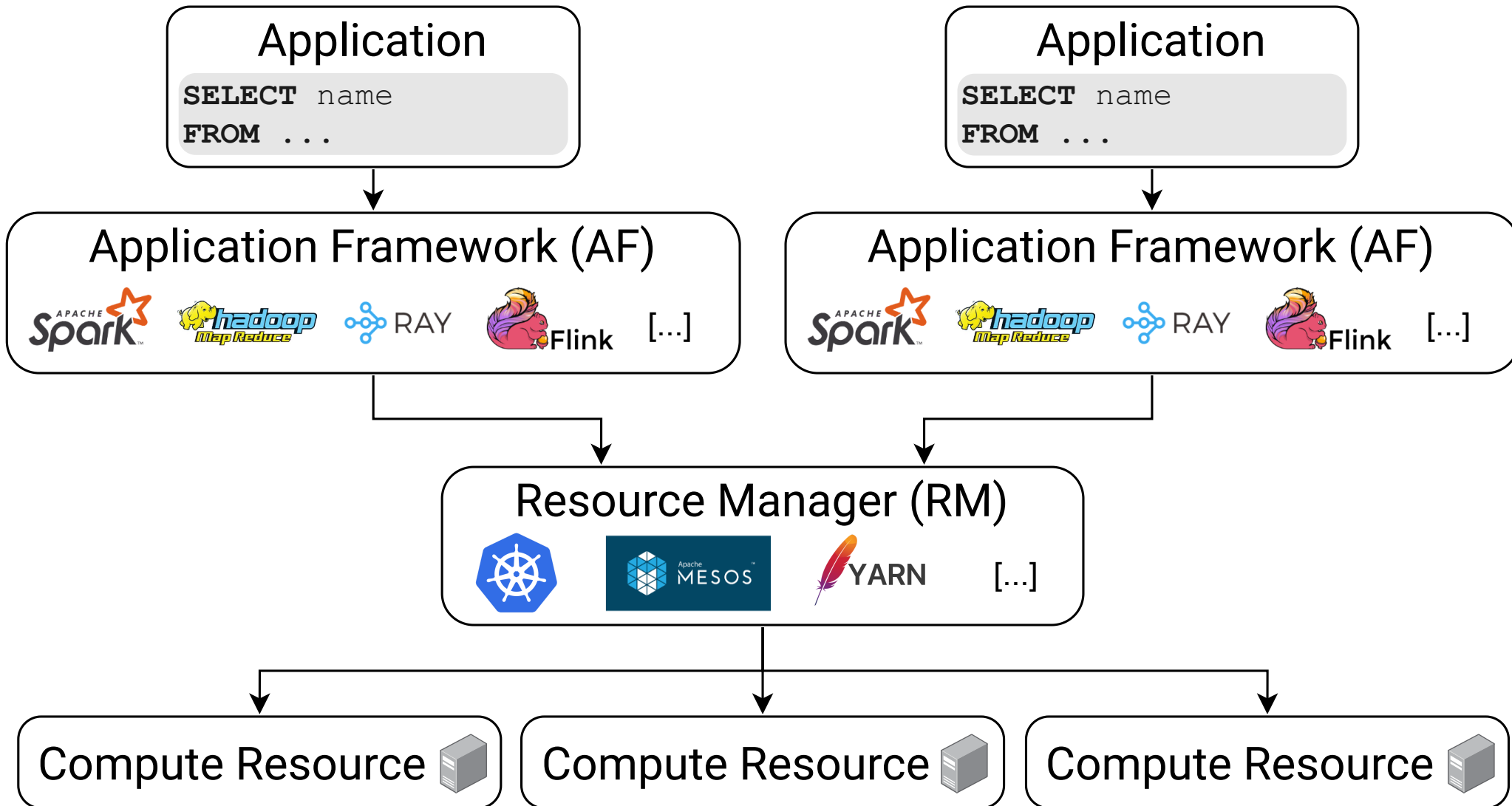
Physics 💡

📺 Entertainment

🤖 AI

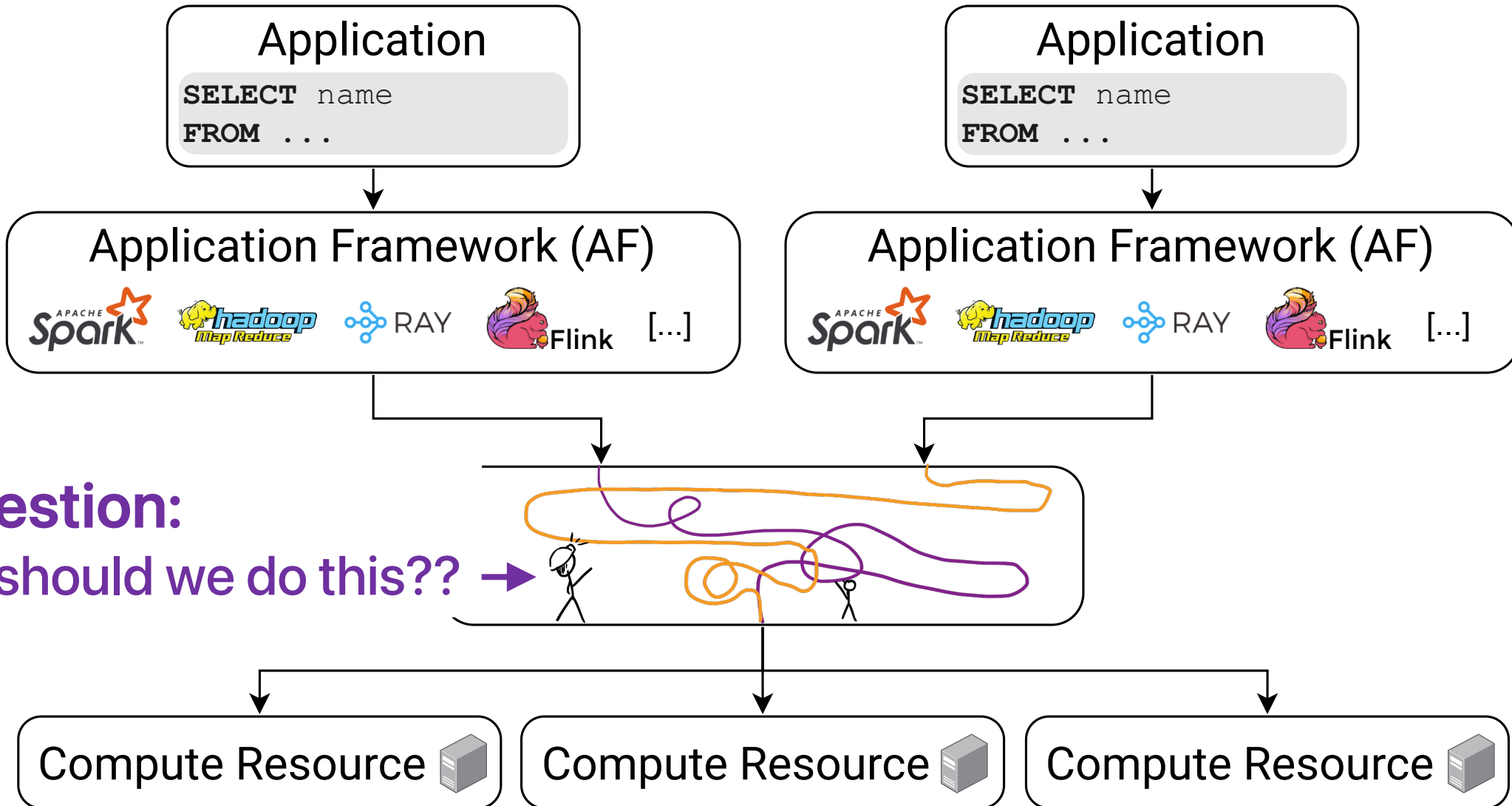








Question:
How should we do this?? ➔



Main RQ: How to evaluate the performance characteristics of resource sharing mechanisms in distributed computing?

using SparkSQL on Kubernetes

RQ1:

How to generate workloads to evaluate the performance characteristics of resource sharing mechanisms?

RQ2:

How to implement an infrastructure framework for automated comparison of resource sharing mechanisms?

RQ3:

What are the performance characteristics of three selected resource sharing mechanisms?

Main RQ: How to evaluate the performance characteristics of resource sharing mechanisms in distributed computing?

using SparkSQL on Kubernetes

⚠ No system exists!

⦿ Answering these research questions helps to address:

- Responsibility,
- Sustainability, and
- Usability

of computer systems, as described in the [CompSys NL Manifesto](#).

RQ1: How to generate workloads to evaluate the performance of resource sharing mechanisms?

Workload:

A (structured) **Collection of Queries on Data.**

Workload: A (structured) **Collection of Queries** on **Data**.[✓]

1. The TPC-DS Dataset

Synthetic dataset, modelling the sales and returns process for a multichannel (stores, catalogs, internet) commercial organization

2. Adapting the TPC-DS Queries

TPC-DS queries are good – but limited in number

Solution:

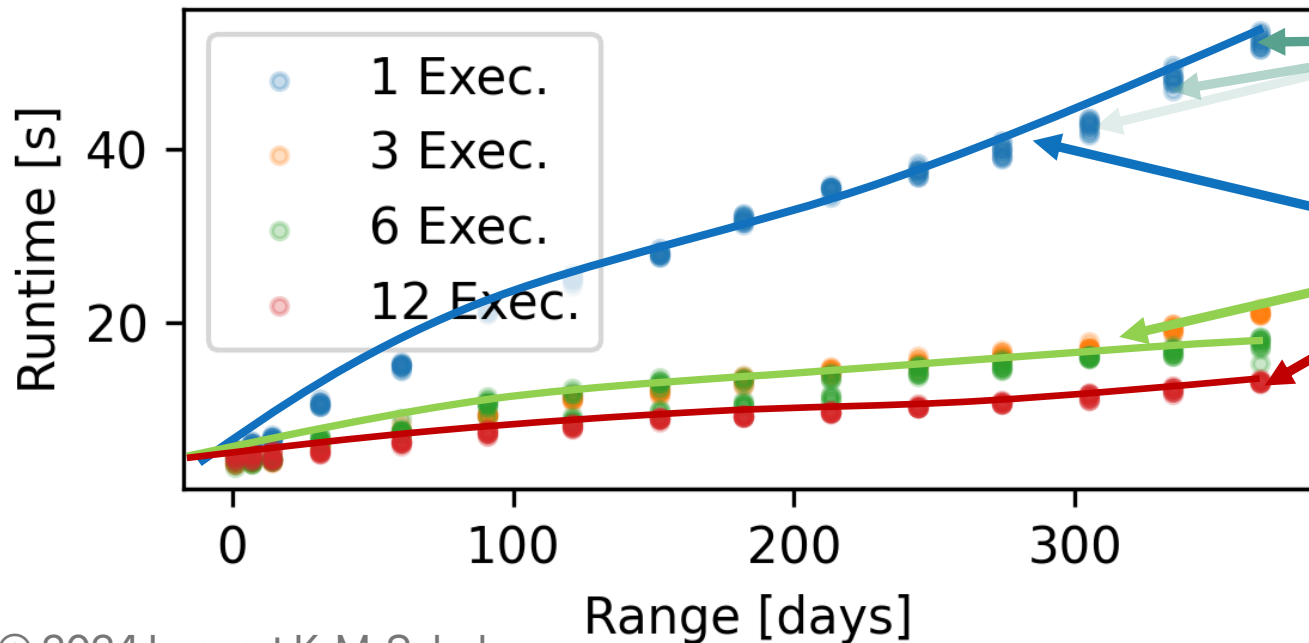
Modify queries into numerous new ones of different scales

Workload: A (structured) **Collection of Queries** on **Data**. ✓

1. The TPC-DS Dataset
2. Adapting the TPC-DS Queries

3. Getting Query Times

Run each query with various scales to get insights into its runtimes



Currently:

Only using measured queries.

Future Work:

Can these relations be modelled?
(→ interpolation & better scaling)

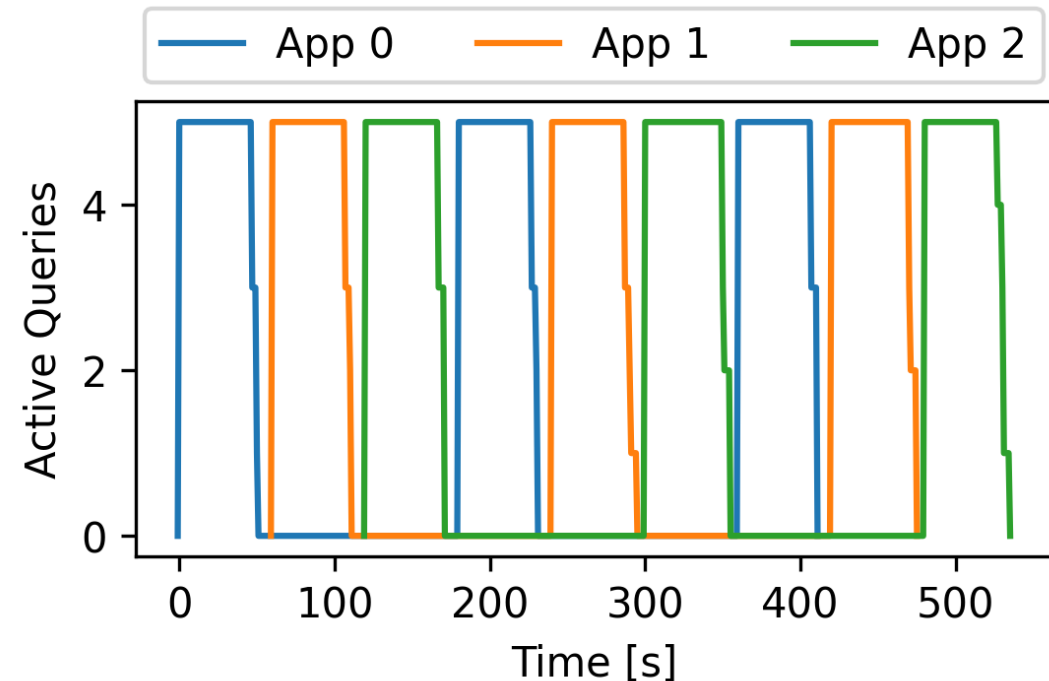
Workload: A (structured) Collection of **Queries**✓ on **Data**✓.

1. The TPC-DS Dataset
2. Adapting the TPC-DS Queries
3. Getting Query Times

4. Building Workloads

Combine the queries into workloads, using the recorded time information:

```
1 workload = create_bursty(  
2     queries,  
3     num_apps=3,  
4     app_offset=60,  
5     burst_count=3,  
6     burst_interval=60 * 3,  
7     ...  
8 )
```



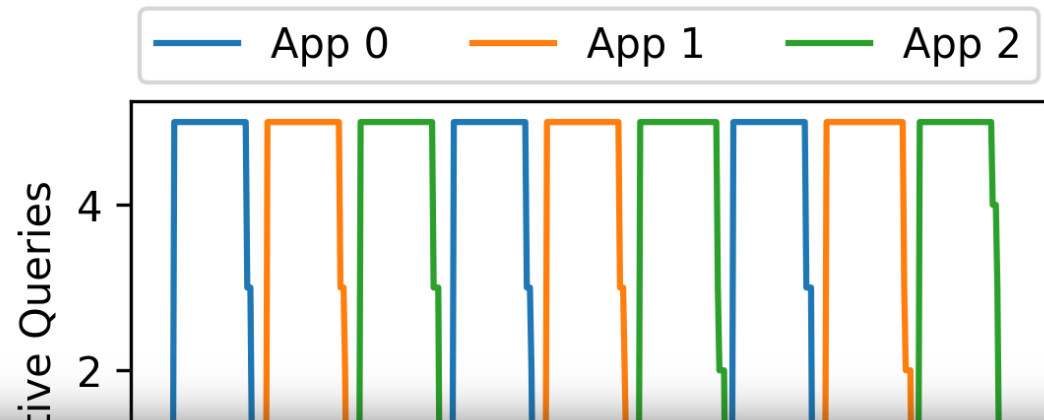
Workload: A (structured) **Collection** of **Queries** on **Data**.

1. The TPC-DS Dataset
2. Adapting the TPC-DS Queries
3. Getting Query Times

4. Building Workloads

Combine the queries into workloads, using the recorded time information:

```
1 workload = create_bursty(  
2     queries,  
3     num_apps=3,  
4     app_offset=60,  
5     burst_count=3,  
6     burst_interval=60 * 3
```

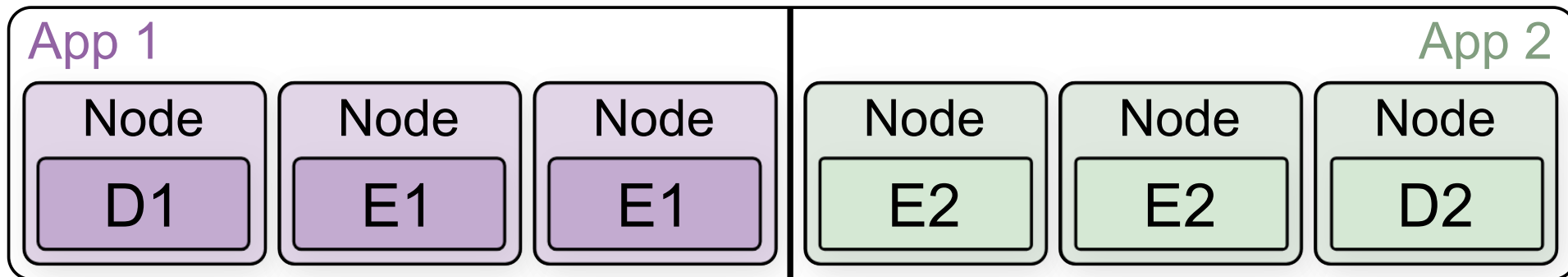


RQ1:

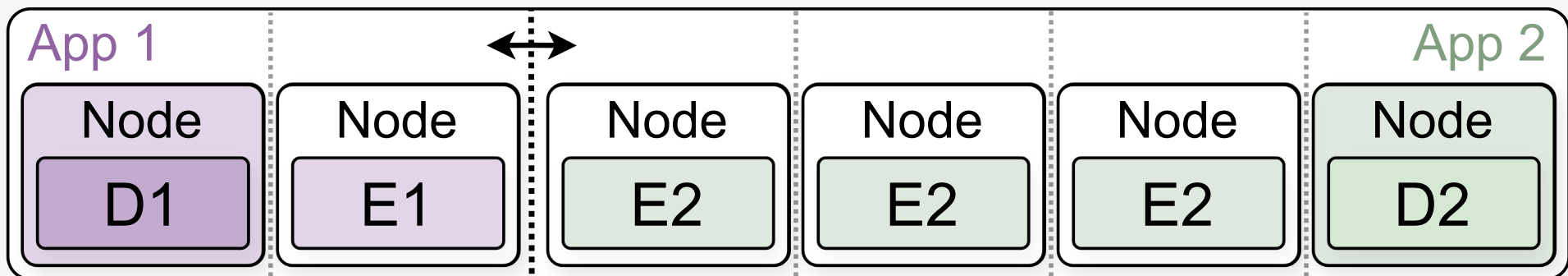
How to generate workloads to evaluate the performance characteristics of resource sharing mechanisms?

Context – Resource Sharing Mechanisms

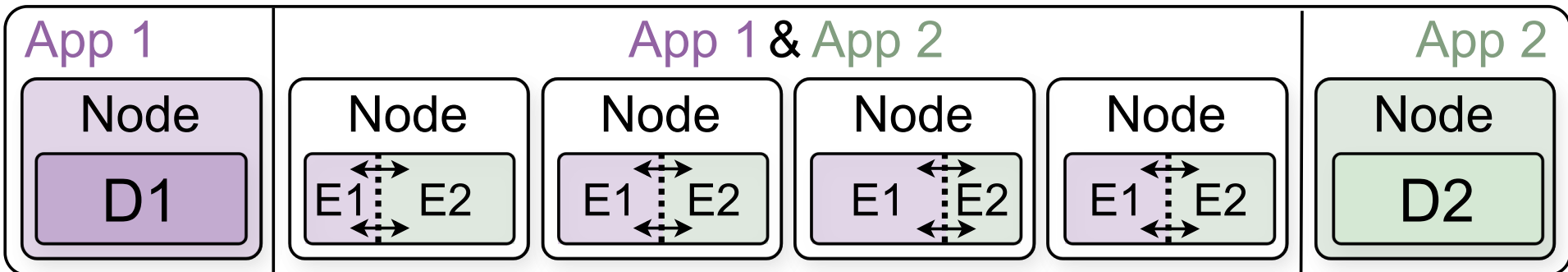
Static
Partitioning



Dynamic
Partitioning



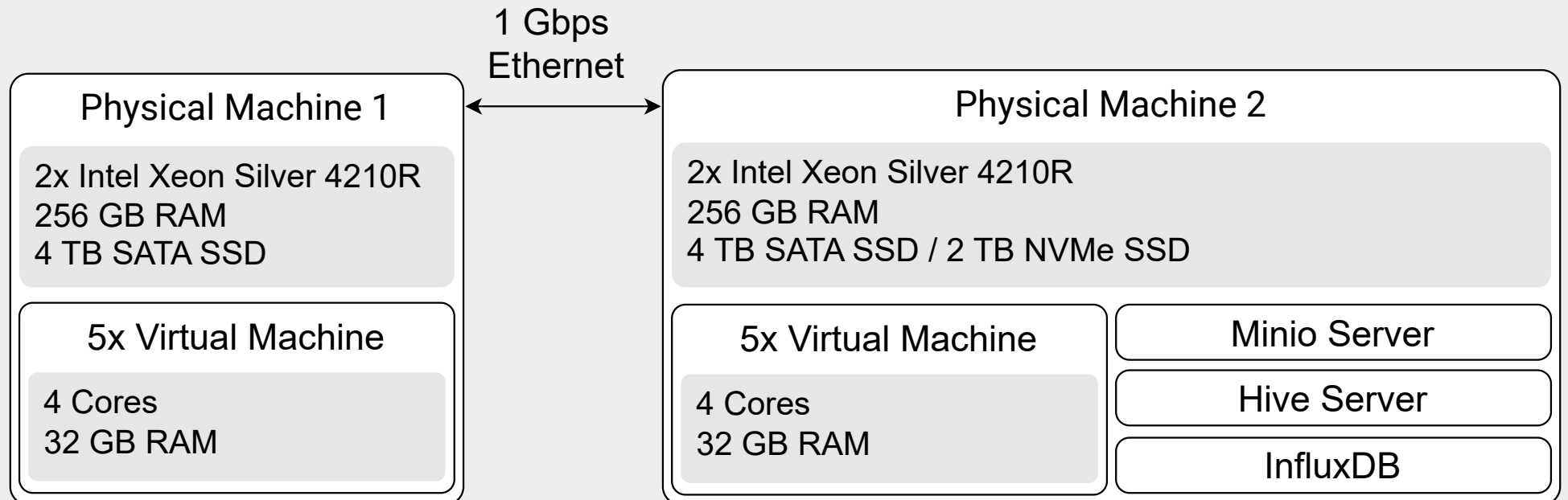
Node-Level
Sharing



RQ3:

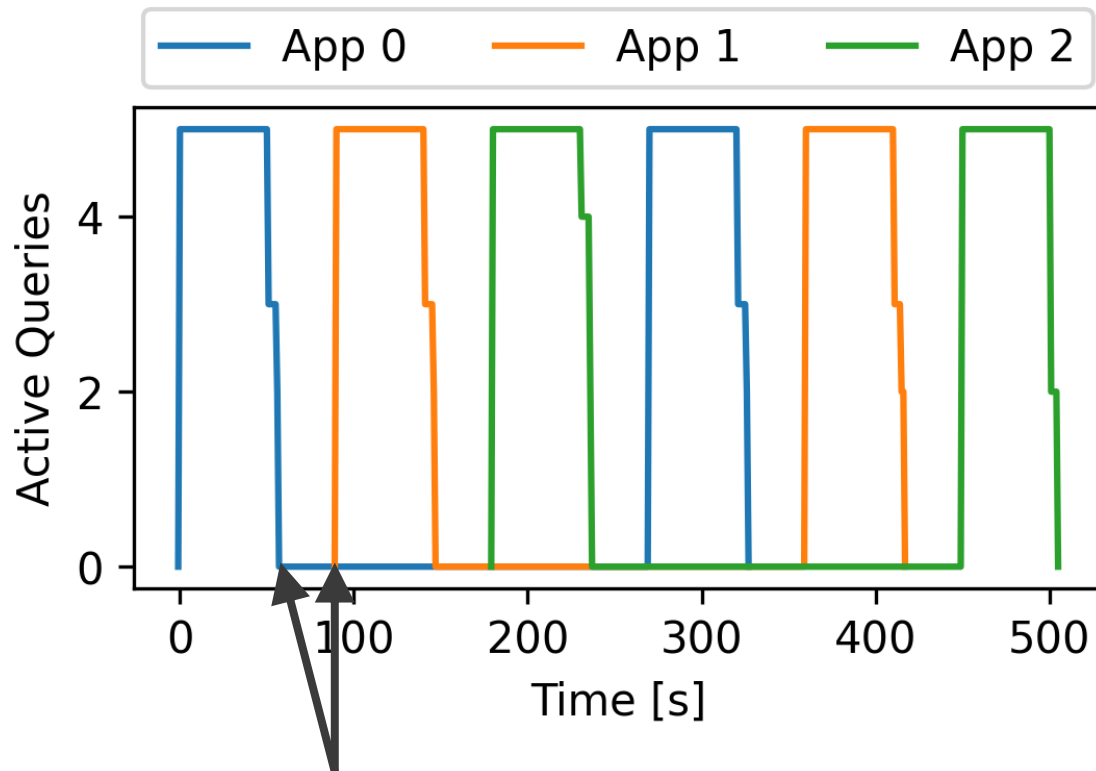
What are the performance characteristics of the selected resource sharing mechanisms?

All experiments conducted on:



Results – Alternating Bursts (with Spacing)

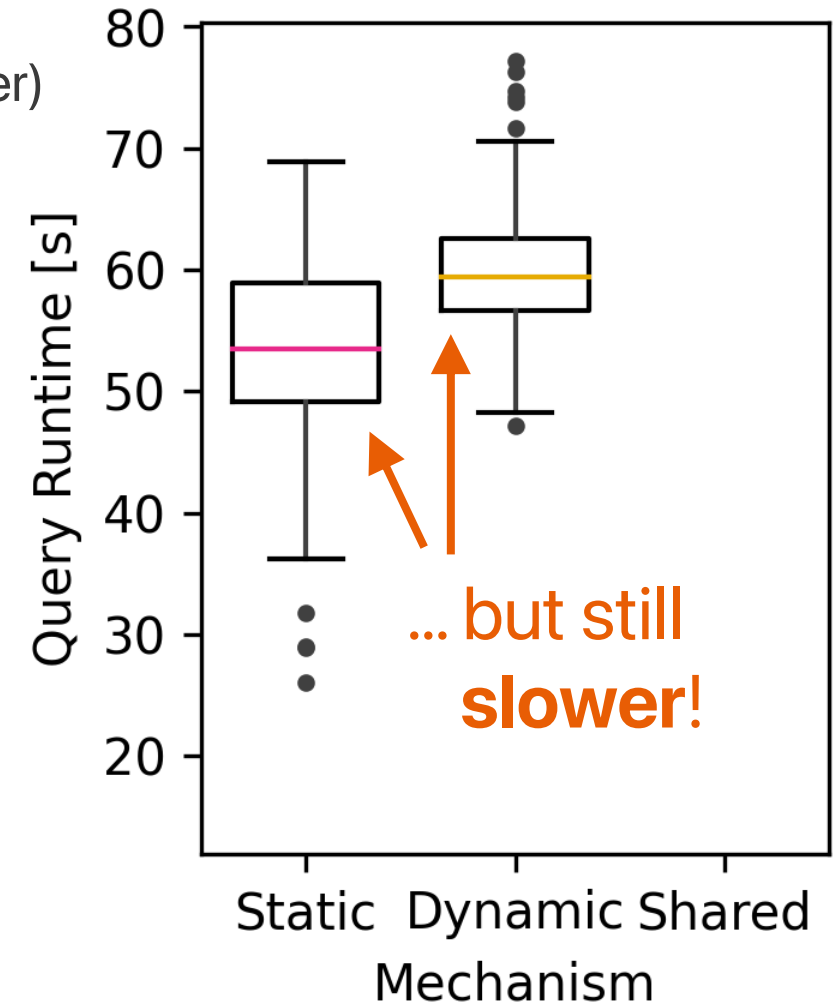
Workload:



30s delay between activity!

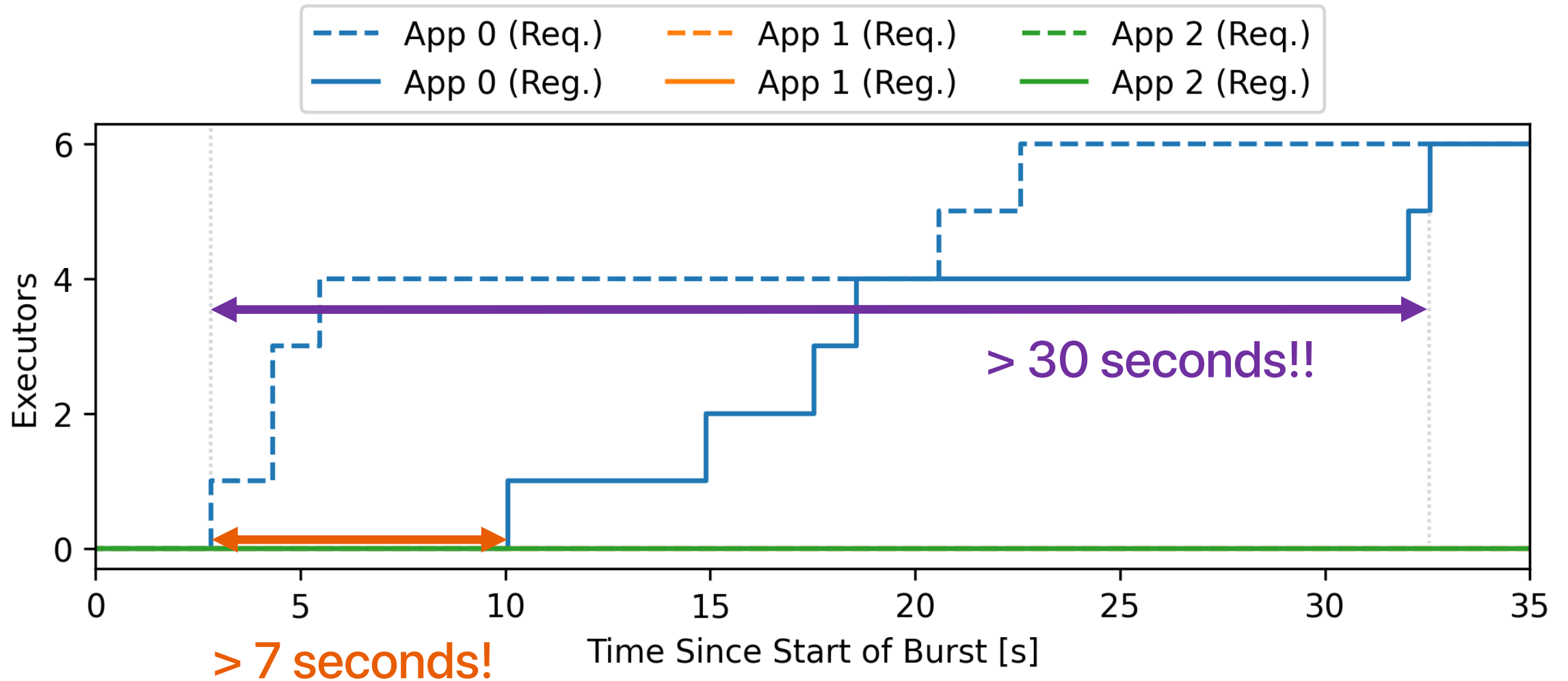
(2x executor timeout → **ideal conditions...**)

Results*: (lower is better)

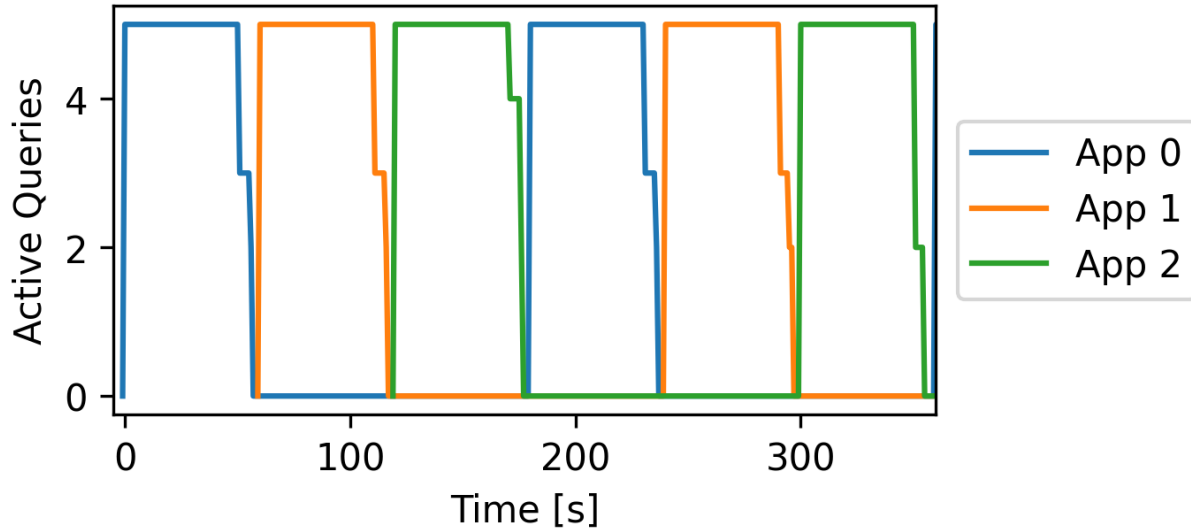


* after a 3-minute warm-up period

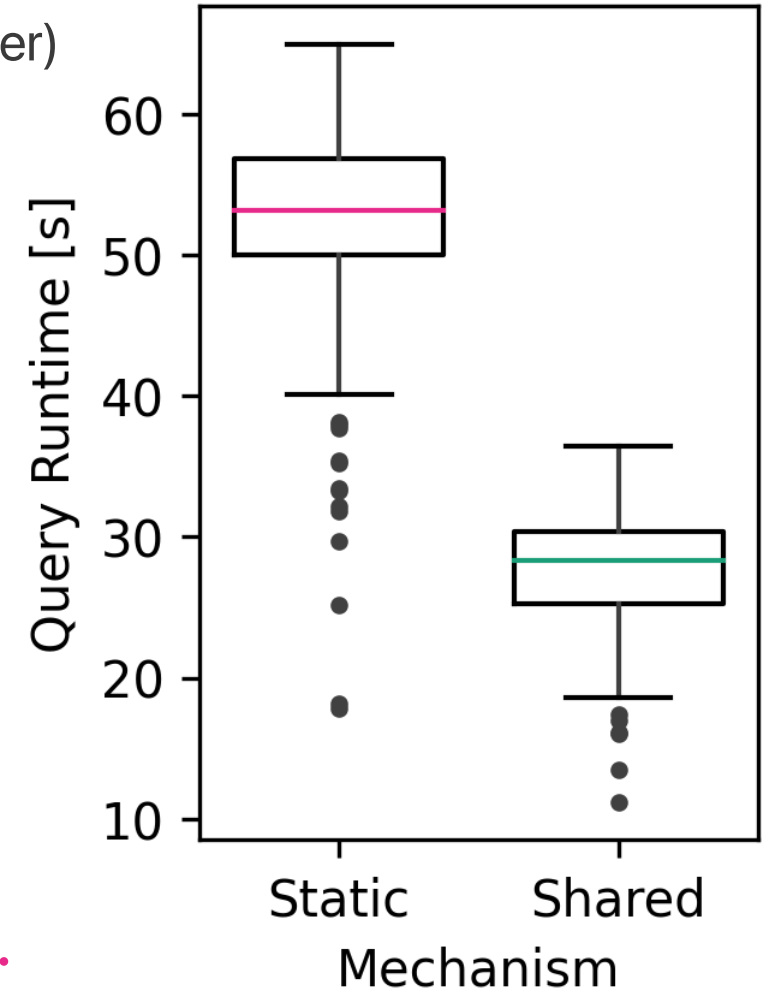
Results – *Dynamic Delays*



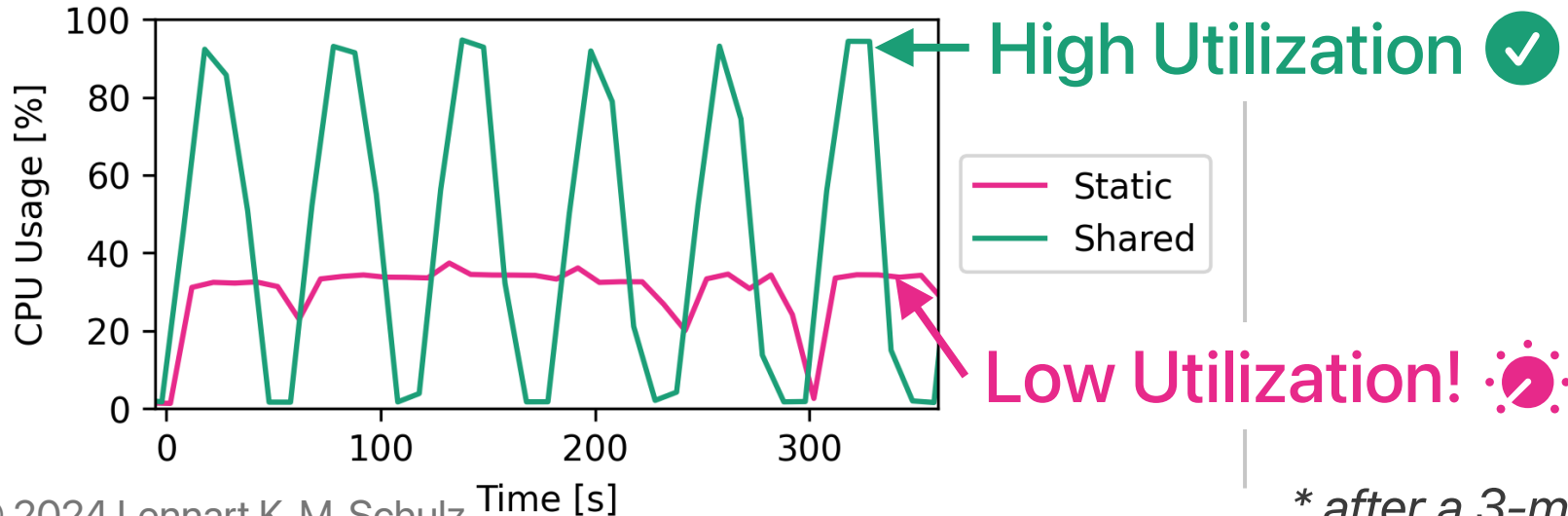
Results – Alternating Bursts



Results*:
(lower is better)

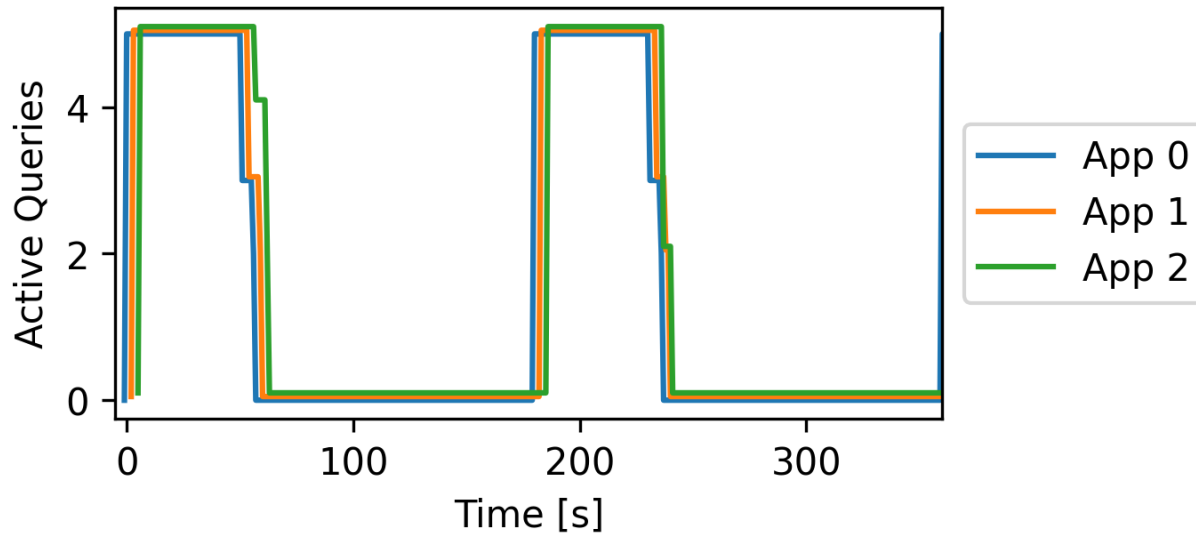


Executor CPU Usage (cluster):

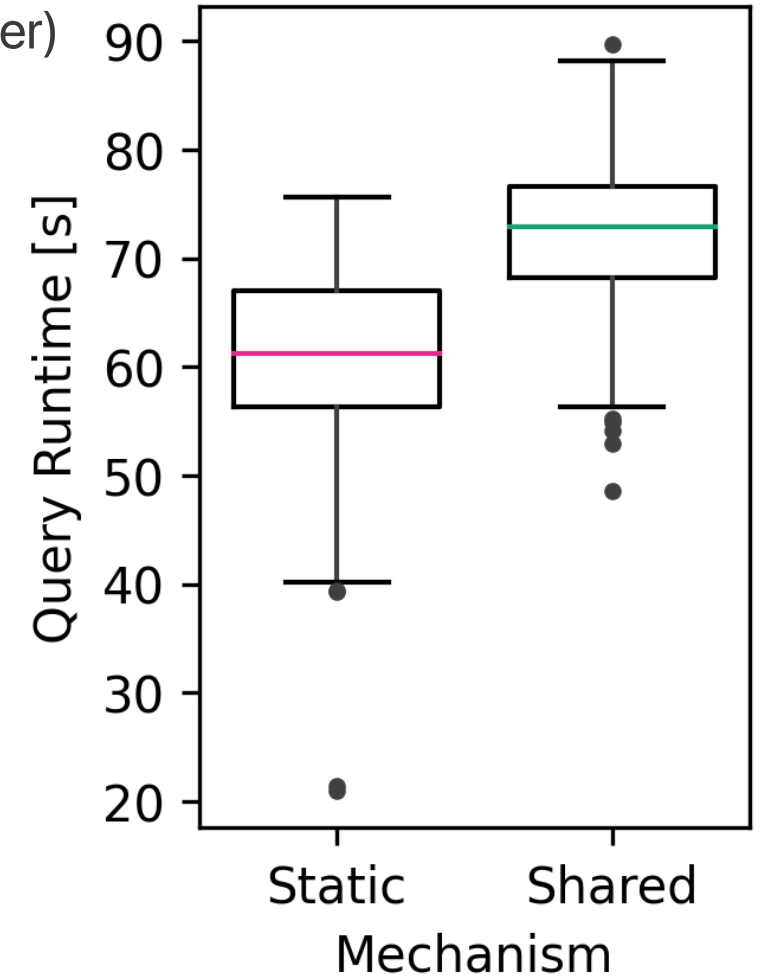


* after a 3-minute warm-up period

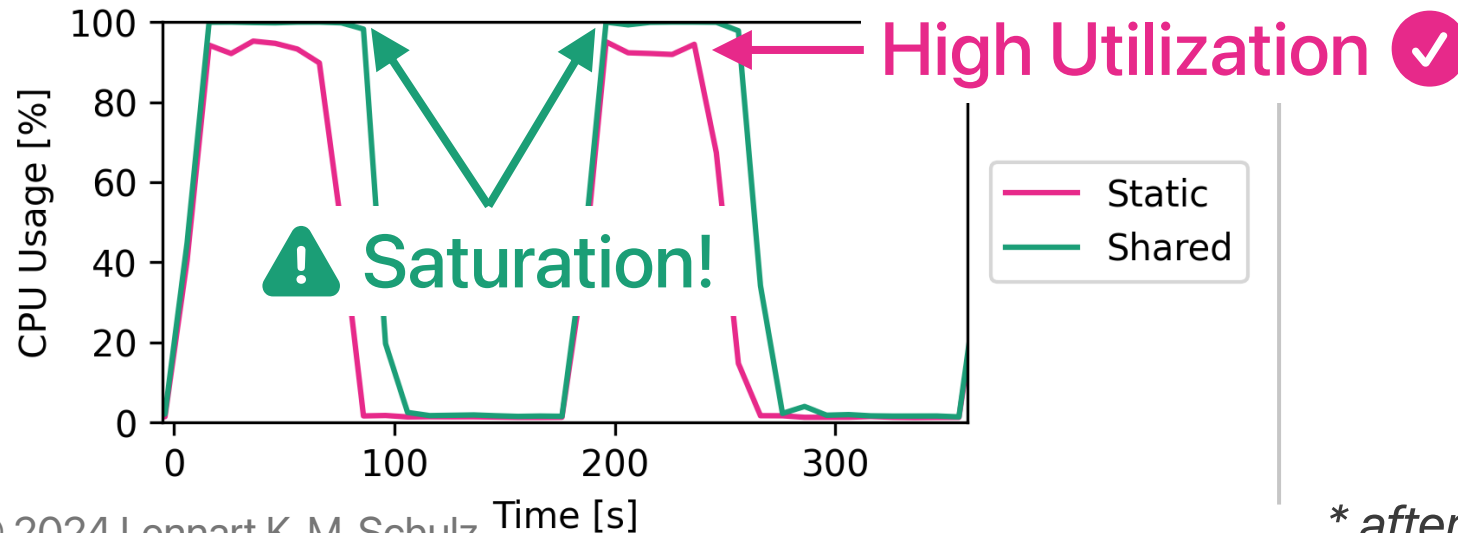
Results – Overlapping Bursts



Results*:
(lower is better)



Executor CPU Usage (cluster):



* after a 3-minute warm-up period

- Resource sharing has become a ubiquitous requirement in clusters.
- ⚠ **No standard system exists** to evaluate resource sharing mechanisms!

🌟 We propose:

1. Workload Generator Design, and
 2. Experiment Infrastructure Framework,
- using which we evaluate the performance characteristics of three resource sharing mechanisms of Spark on Kubernetes.



* <https://github.com/lkm-schulz/FraPChaRSM>

Extra Material

Workload: A (structured) **Collection of Queries on Data**.[✓]

1. The TPC-DS Dataset

2. Adapting the TPC-DS Queries

TPC-DS queries are good – but limited in number and inflexible.

Solution: modify given queries into numerous new ones of different scales by changing the range of included data:

```
1 SELECT sr_customer_sk
2 FROM store_returns, date_dim
3 WHERE d_year = 2000 AND ...
```



```
1 SELECT sr_customer_sk
2 FROM store_returns, date_dim
3 WHERE ($DATERANGE$) AND ...
```



```
WHERE (d_year = 2000 and d_moy = 01) AND ...
```

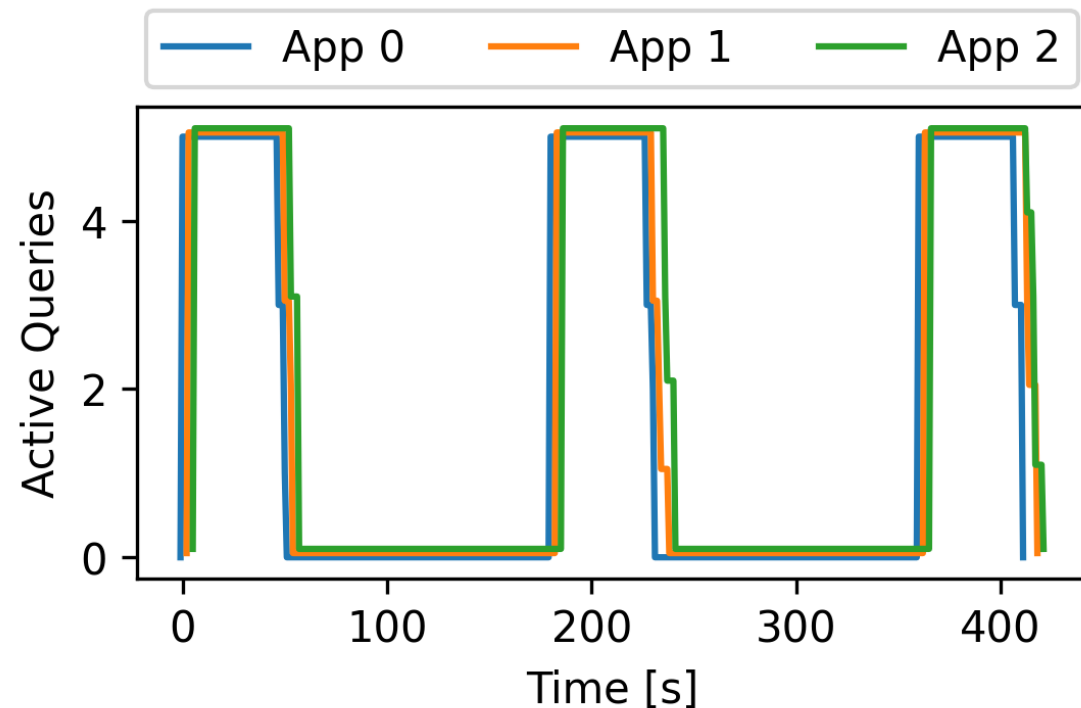
```
WHERE (d_year = 2000 or d_year = 2001) AND ...
```

```
...
```

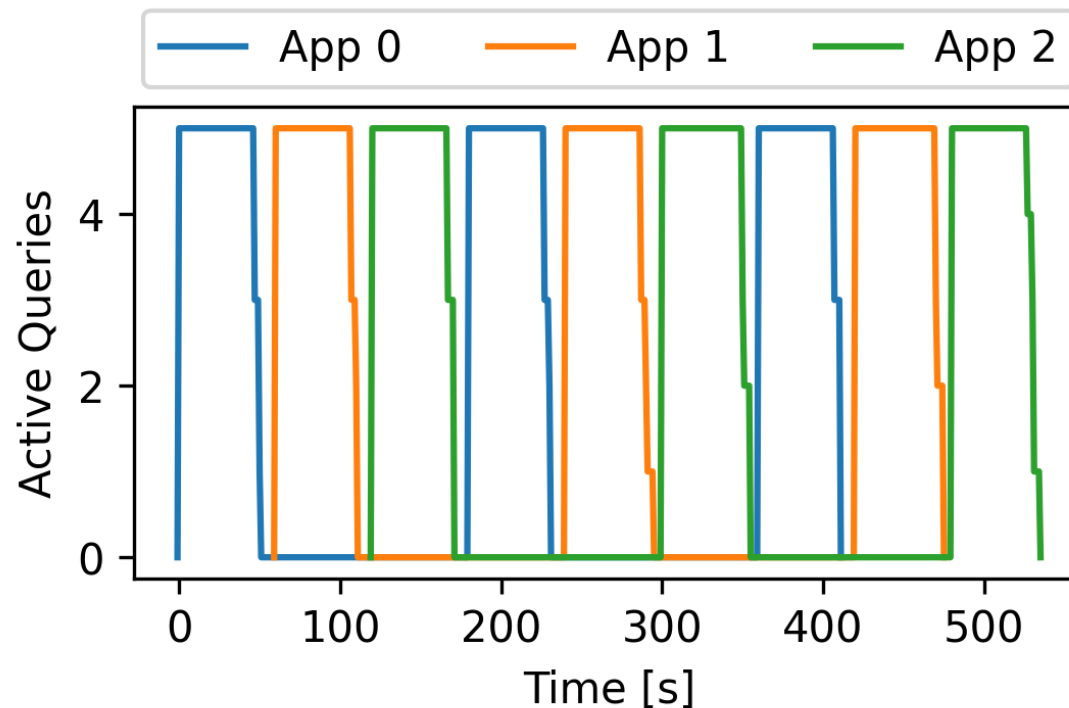
Workload: A (structured) **Collection** of **Queries** on **Data**.

1. The TPC-DS Dataset
2. Adapting the TPC-DS Queries
3. Getting Query Times
4. Building Workloads

Overlapping Bursts:



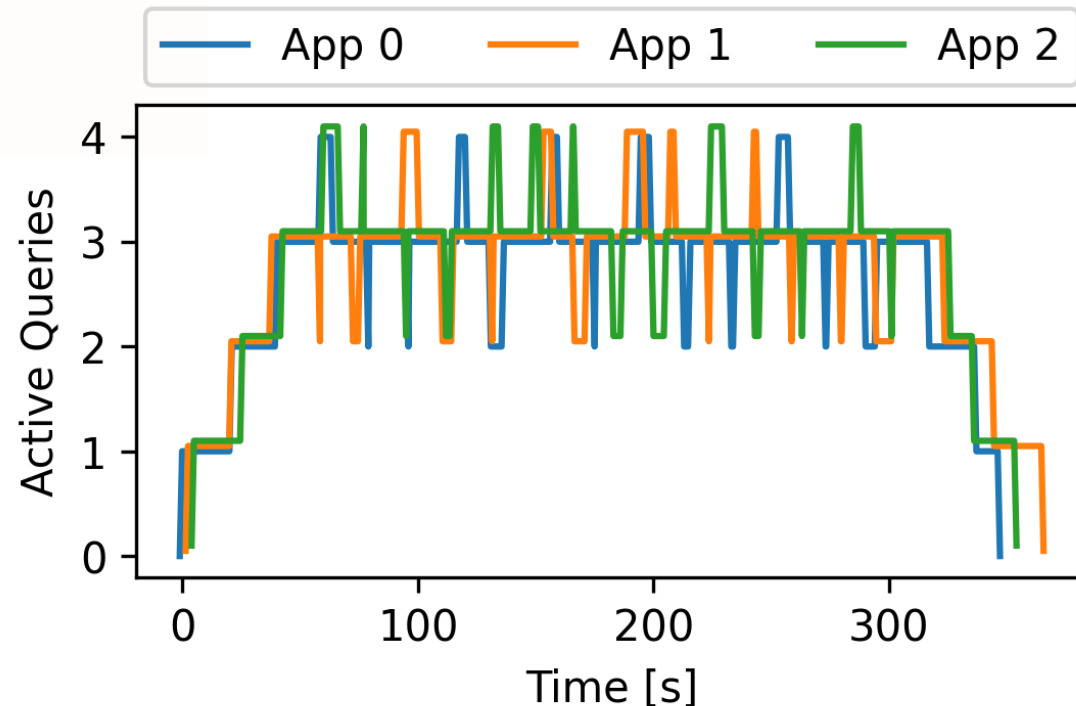
Alternating Bursts:



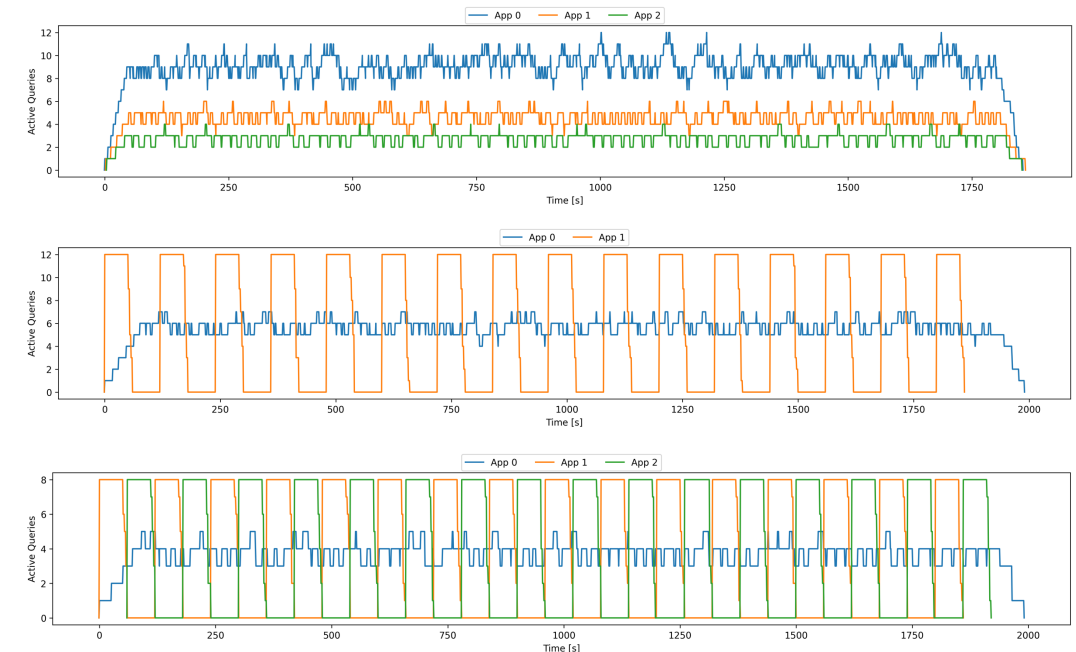
Workload: A (structured) **Collection** of **Queries** on **Data**.

1. The TPC-DS Dataset
2. Adapting the TPC-DS Queries
3. Getting Query Times
4. Building Workloads

Constant Load:



And many more...

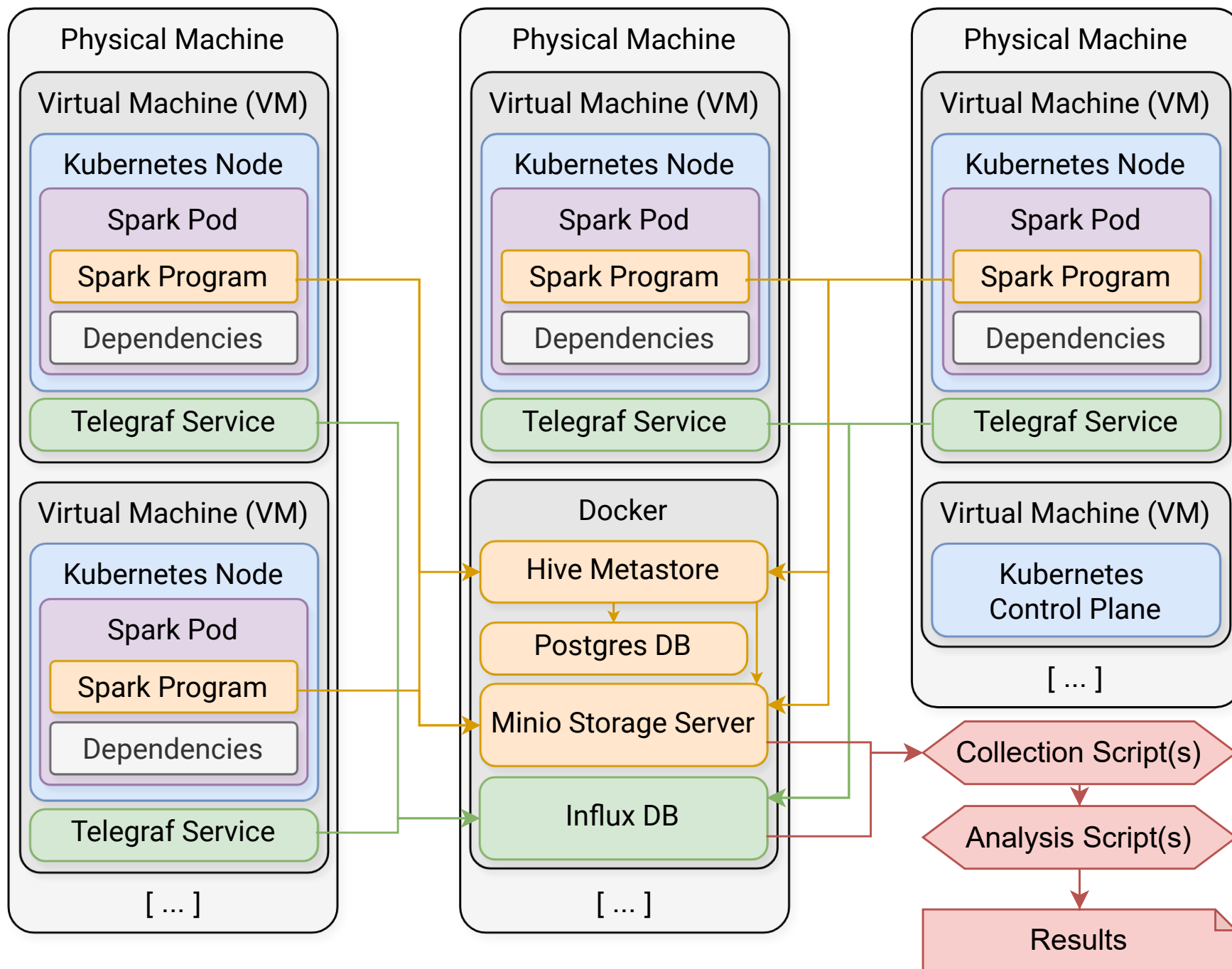


Infra- structure

- ✓ reproducible
- ✓ automated
- ✓ extensible

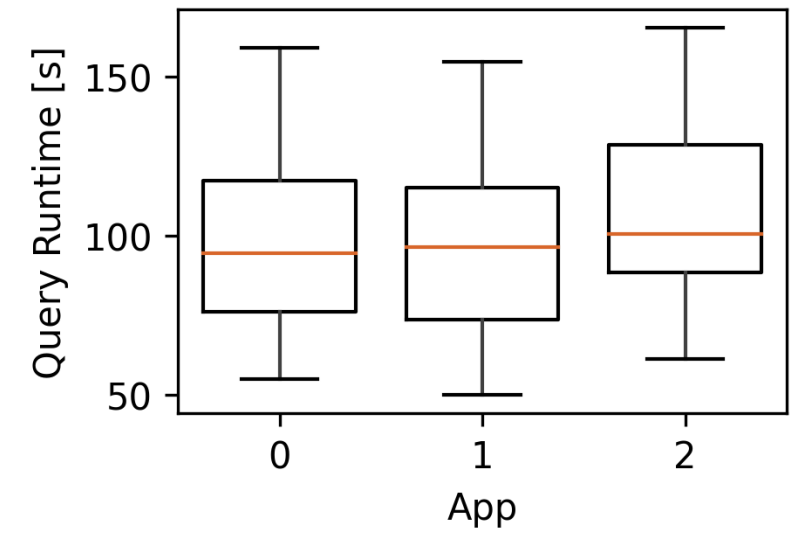
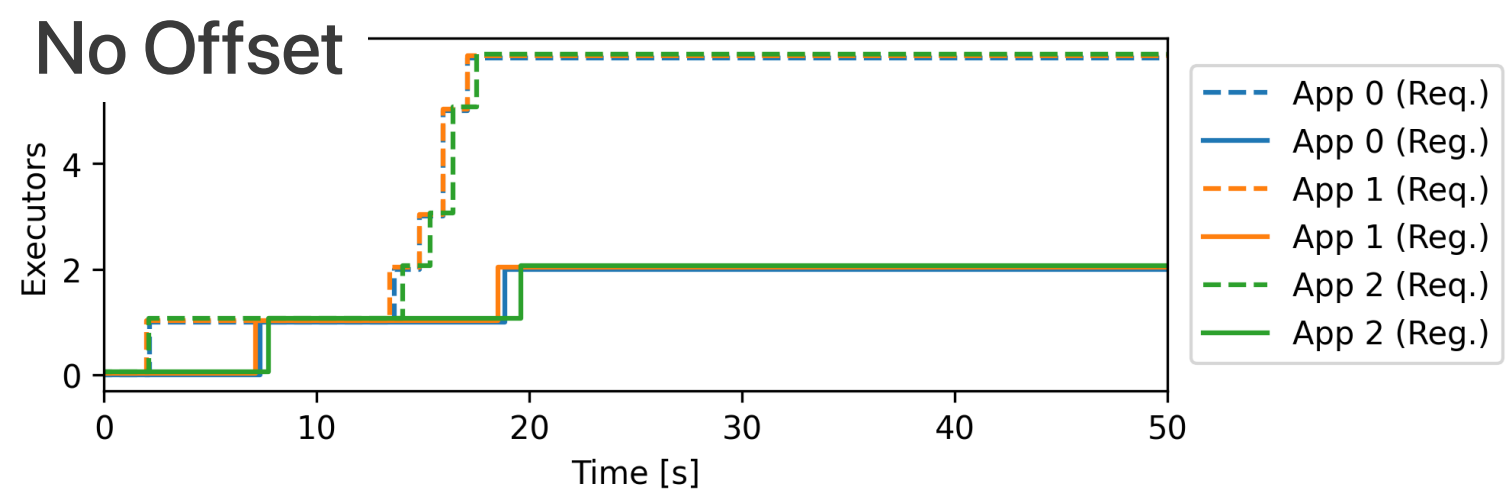
@Large Research
Massivizing Computer Systems

© 2024 Lennart K. M. Schulz

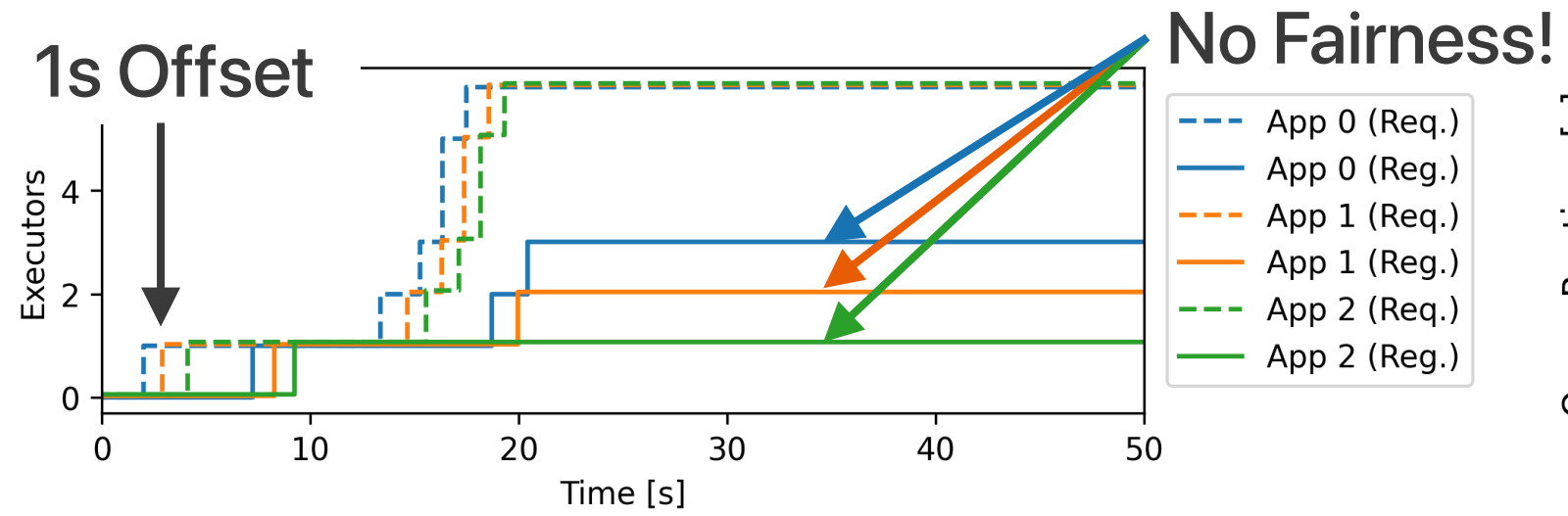


Results – *Dynamic (Un)Fairness*

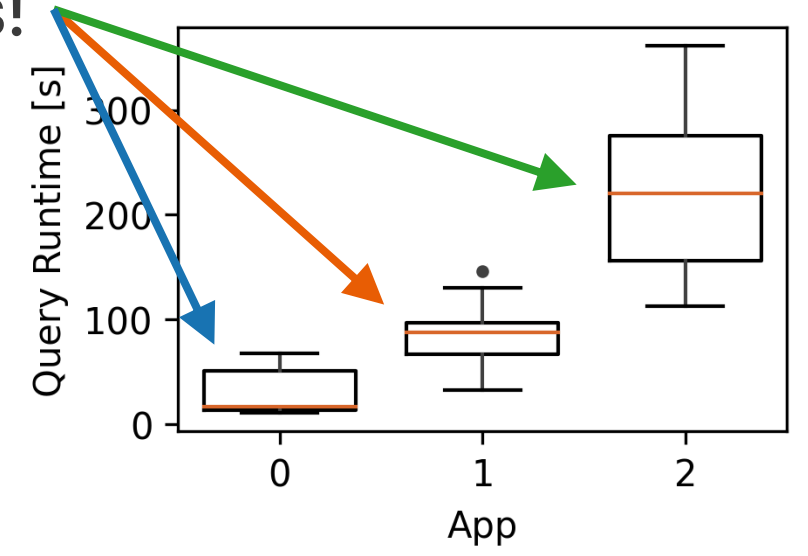
No Offset



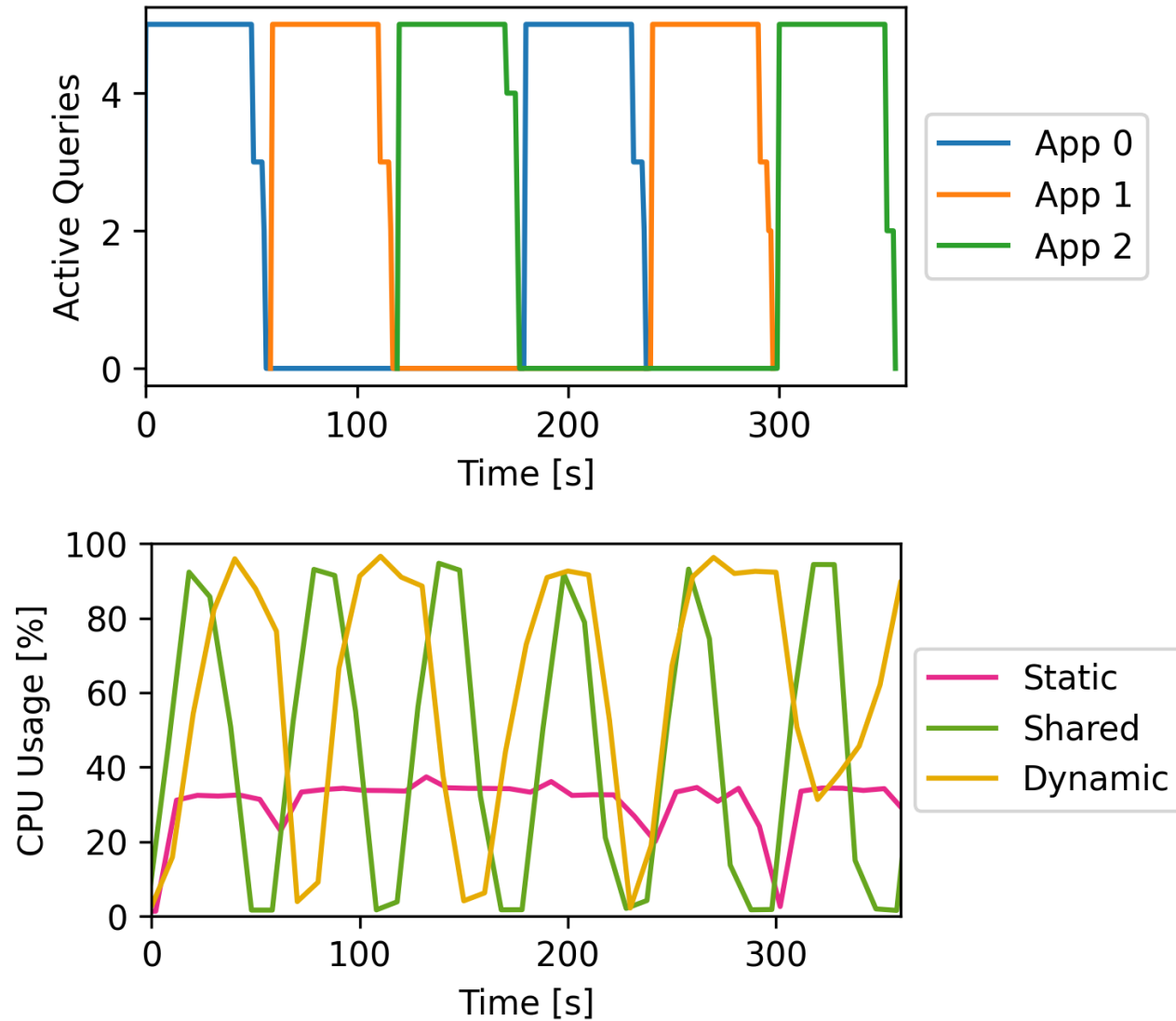
1s Offset



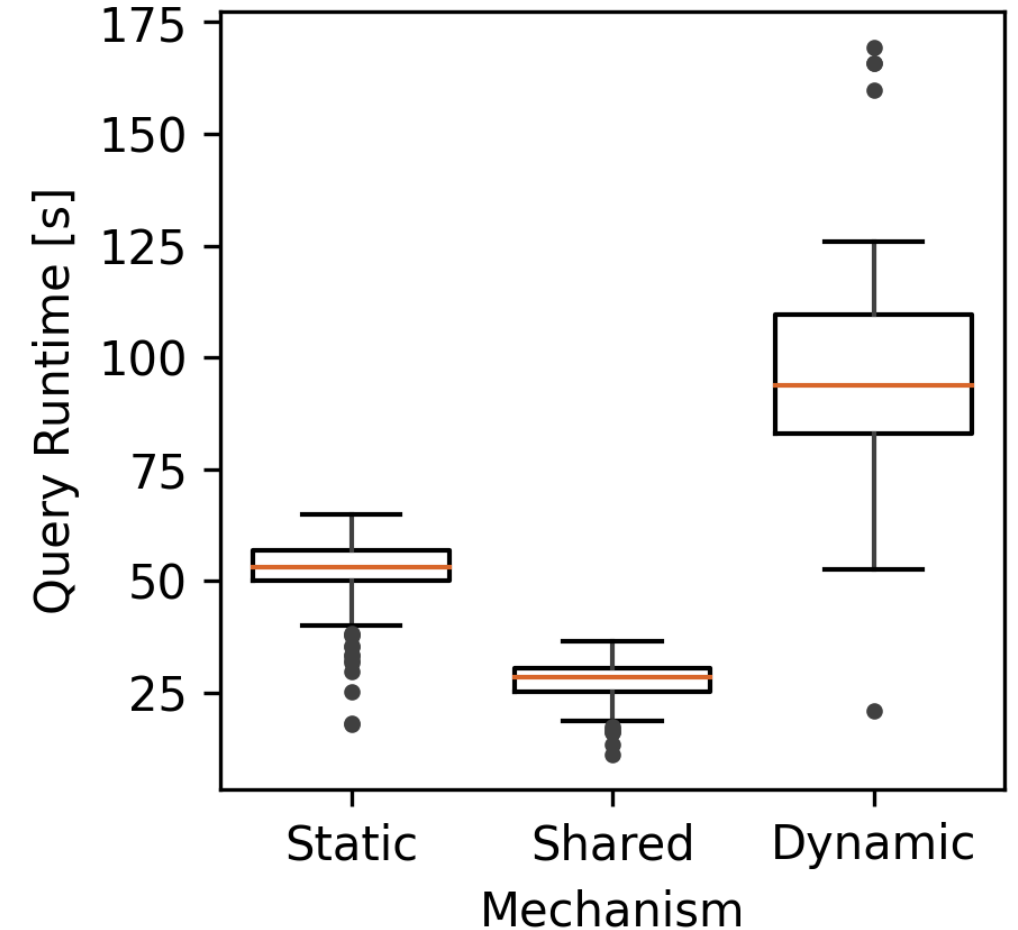
No Fairness!



Results – Alternating Bursts



Results*:



* after a 3-minute warm-up period